

Microservices — Module 01: Communication Patterns

Complete Production Study Guide

Swati Chinchole | Study Session: May 9, 2026

Series: Microservices Production Mastery — Module 1 of 6

Next: Module 02 — Data Patterns (15 patterns)

Module Checklist

#	Topic	Status
1	Synchronous vs Asynchronous	✓ Done
2	REST — internals + production pitfalls	✓ Done
3	gRPC — what it is + when it beats REST	✓ Done
4	Kafka — internals + 5 production killers	✓ Done
5	RabbitMQ — how it differs from Kafka	✓ Done
6	API Gateway — the front door	✓ Done
7	Service Mesh — internal traffic control	✓ Done
8	8 Fallacies of Distributed Systems	✓ Done (previous session)

#1 — Synchronous vs Asynchronous

The Core Decision

"Does the caller need the result immediately to continue?"

YES → Synchronous (REST/gRPC)

NO → Asynchronous (Kafka/RabbitMQ)

How They Work

Synchronous:

Service A calls Service B and WAITS
Service A is blocked until Service B responds
Like a phone call – you wait until other person answers

Asynchronous:

Service A sends a message and MOVES ON
Does not wait – doesn't know when B will process it
Like WhatsApp – you send and continue your day

Real BFSI Decision Table

Operation	Sync or Async?	Why
Check account balance	Sync	User is waiting on screen
Validate OTP at login	Sync	Cannot proceed without result
Process payment	Sync	User needs confirmation NOW
Send SMS after payment	Async	User doesn't wait for SMS
Update credit score	Async	Background job, no urgency
Generate PDF statement	Async	Can take time, email when ready
Fraud detection alert	Async	Log it, investigate later

The Golden Rule

Everything the USER is actively waiting for → Synchronous
Everything that happens in the BACKGROUND → Asynchronous

What Goes Wrong When You Choose Wrong

Sync where Async should be (SMS via sync call):

Payment Service calls Notification Service (SYNC)
SMS Gateway goes down → Notification Service times out
Payment Service times out → User sees "Payment Failed"
But money WAS debited! Customer's money gone silently 🤖

Fix: Make SMS notification ASYNC via Kafka
Payment processes → returns success immediately
SMS sent in background → SMS failure invisible to user

Async where Sync should be (balance check via async):

Payment Service sends debit via Kafka (ASYNC)
Returns "Payment Successful" to user immediately
Account Service processes → INSUFFICIENT BALANCE ✖
Shown success → actual failure = worst UX possible

Fix: Balance check must be SYNC
Get confirmation before telling user anything

The Decision Framework

```
Ask: "Does caller need result to proceed?"      → YES = Sync
Ask: "Is this background/notification operation?" → YES = Async
Ask: "What if receiving service is slow/down?"
  Caller blocked/fails → consider making it Async
Ask: "Does order/sequence matter?"
  YES + needs confirmation → Sync
  YES + same entity → Async with Kafka partition key
```

#2 — REST in Microservices

How REST Works Under the Hood

Every REST call goes through 6 steps:

```
Step 1: DNS Resolution      → "account-service" → IP 172.18.0.5
Step 2: TCP Connection     → Three-way handshake (1-3ms)
Step 3: HTTP Request sent  → Headers + JSON body
Step 4: Service B processes → Deserialize → logic → DB → serialize
Step 5: HTTP Response      → 200 OK + JSON body
Step 6: Connection close   → or kept alive (connection pool)
```

Each step can fail. Each step adds latency.

The 4 Silent Killers of REST in Production

Killer 1 — No Connection Pooling

```
// BAD – new TCP connection every request (3ms wasted per call)
RestTemplate restTemplate = new RestTemplate();

// GOOD – reuse existing connections
@Bean
public RestTemplate restTemplate() {
    PoolingHttpClientConnectionManager cm = new PoolingHttpClientConnectionManager(
        cm.setMaxTotal(200);
        cm.setDefaultMaxPerRoute(50);
        CloseableHttpClient httpClient = HttpClients.custom()
            .setConnectionManager(cm).build();
    return new RestTemplate(new HttpComponentsClientHttpRequestFactory(httpClient));
}
```

Killer 2 — No Timeout Configuration

```
// BAD – waits forever
restTemplate.get("http://fraud-service/check/" + id);

// GOOD – fail fast
```

```

HttpComponentsClientHttpRequestFactory factory =
    new HttpComponentsClientHttpRequestFactory();
factory.setConnectTimeout(1000); // 1 second to connect
factory.setReadTimeout(3000);    // 3 seconds for response

// BFSI timeout rules:
// Internal service calls    → 2-3 seconds max
// Payment gateway calls     → 5-10 seconds max
// Credit bureau calls       → 10-15 seconds max
// NEVER more than 30 seconds

```

Killer 3 — Wrong HTTP Status Codes

```

// BAD – always 200, error buried in body
return new Response("error", "Account not found"); // HTTP 200 ← WRONG

// GOOD – use correct HTTP codes
return ResponseEntity.notFound().build();           // HTTP 404
return ResponseEntity.status(403).build();          // HTTP 403
return ResponseEntity.internalServerError().build(); // HTTP 500

```

Status Code Cheat Sheet:

Code	Meaning	When to use
200	OK	Successful GET, PUT
201	Created	Successful POST
400	Bad Request	Invalid input
401	Unauthorized	Not logged in
403	Forbidden	No permission
404	Not Found	Resource missing
409	Conflict	Duplicate / idempotency
422	Unprocessable	Business rule violated
429	Too Many Requests	Rate limit exceeded
500	Server Error	Your service crashed
503	Unavailable	Downstream dependency down

Killer 4 — No Correlation ID

```

// API Gateway generates unique ID per request
String correlationId = UUID.randomUUID().toString();
headers.add("X-Correlation-ID", correlationId);

// Every service logs with this ID
log.info("[{}] Processing payment for account: {}", correlationId, accountId);

```

```
// Every service passes it downstream
headers.add("X-Correlation-ID", request.getHeader("X-Correlation-ID"));

// Result: search abc-123 → see complete journey across all services
```

#3 — gRPC

What's Wrong With REST for Internal Services?

```
REST uses HTTP/1.1 + JSON:
  JSON is text → large payload → slow to parse
  HTTP/1.1 → one request per connection (no multiplexing)
  No strict contract → field names can change silently
  No streaming support
```

What is gRPC?

gRPC = Google Remote Procedure Call

- HTTP/2 instead of HTTP/1.1
- Protobuf (binary) instead of JSON
- Strict contracts via .proto files
- Code auto-generated for any language

How It Works

Step 1 — Define contract in .proto file:

```
syntax = "proto3";
service PaymentService {
  rpc ProcessPayment (PaymentRequest) returns (PaymentResponse);
}
message PaymentRequest {
  string account_id = 1;
  double amount     = 2;
  string currency   = 3;
}
message PaymentResponse {
  string payment_id = 1;
  string status     = 2;
}
```

Step 2 — Server implements:

```
@GrpcService
public class PaymentServiceImpl extends PaymentServiceGrpc.PaymentServiceImplBase
    @Override
```

```

    public void processPayment(PaymentRequest request,
                               StreamObserver<PaymentResponse> observer) {
        PaymentResponse response = PaymentResponse.newBuilder()
            .setPaymentId(UUID.randomUUID().toString())
            .setStatus("SUCCESS")
            .build();
        observer.onNext(response);
        observer.onCompleted();
    }
}

```

Step 3 — Client calls like local method:

```

PaymentResponse response = paymentClient.processPayment(
    PaymentRequest.newBuilder()
        .setAccountId("ACC123")
        .setAmount(500.00)
        .setCurrency("INR")
        .build()
);

```

REST vs gRPC Comparison

	REST	gRPC
Protocol	HTTP/1.1	HTTP/2
Data Format	JSON (text)	Protobuf (binary)
Payload Size	Large	3–5x smaller
Speed	Baseline	5–10x faster
Contract	Optional	Mandatory (.proto)
Code Generation	Manual	Automatic
Streaming	✗ No	✓ Yes (4 types)
Browser Support	✓ Yes	✗ No (needs proxy)

4 Communication Types in gRPC

1. Unary → 1 request → 1 response (like REST)
2. Server Streaming → 1 request → many responses (live updates)
3. Client Streaming → many requests → 1 response (batch upload)
4. Bidirectional → both stream simultaneously (real-time fraud monitoring)

When to Use What

Use REST when:

- Public API (mobile, browsers, third parties)
- Human readability needed for debugging
- Simple CRUD operations

Use gRPC when:

- Internal service-to-service communication
- High performance needed (payments, trading)
- Streaming data (live updates, batch)
- Strict contracts needed

Real world: REST for external, gRPC for internal
(Razorpay, Google, Netflix all follow this pattern)

Production Gotcha — Load Balancing with gRPC

gRPC uses ONE long-lived HTTP/2 connection
Traditional load balancer sees 1 connection → sends all to Server 1
Server 2, 3 idle → Server 1 overloaded

Fix: Use gRPC-aware load balancer

- Kubernetes with Istio (load balance at request level)
- Client-side load balancing with service discovery

#4 — Apache Kafka

What Kafka Is

Kafka is a **distributed commit log** — a middle layer that decouples producers from consumers.

WITHOUT Kafka:

Payment Service directly calls Notification, Fraud, Analytics, Audit
Tight coupling, payment knows about everyone

WITH Kafka:

Payment Service → Kafka Topic ← All consumers subscribe
Payment Service knows about NOBODY
Any new consumer? Just subscribe to topic, no code change

Core Concepts

Topic = database table for events

"payment-events" → all payment events
"account-events" → all account events

Partition = lanes on a highway (parallelism)

```
Topic: "payment-events" (4 partitions)
Partition 0: [msg1, msg5, msg9 ...]
Partition 1: [msg2, msg6, msg10 ...]
Partition 2: [msg3, msg7, msg11 ...]
Partition 3: [msg4, msg8, msg12 ...]
```

Ordering guaranteed WITHIN one partition
4 consumers can read simultaneously

Offset = page number per message

```
Consumer remembers: "read up to offset 47"
If consumer crashes → restarts from offset 48
Zero message loss
```

Consumer Group = group of service instances

```
notification-group: Instance 1 reads Partition 0
                    Instance 2 reads Partition 1
fraud-group:        gets ALL messages independently
analytics-group:    gets ALL messages independently
```

How a Message Flows

```
Step 1: Producer sends with KEY (accountId)
Step 2: Kafka hashes key → assigns partition
        hash("ACC123") % 4 = Partition 2
        All messages for ACC123 → always Partition 2 (ordering guaranteed)
Step 3: Broker writes to disk (append-only, 1M msg/sec)
Step 4: Consumer polls: "messages since offset 47?"
Step 5: Consumer processes → commits offset
        Crash and restart → reads from committed offset (no loss)
```

Replication — How Kafka Never Loses Data

```
Every partition: 1 Leader + 2 Replicas (replication factor = 3)
Broker 1: LEADER   ← producers write here
Broker 2: Replica
Broker 3: Replica
```

Broker 1 crashes → Broker 2 becomes Leader
Zero data loss, happens in seconds

```
// Producer config for BFSI – guaranteed delivery
props.put("acks", "all");           // wait for ALL replicas
```



```
props.put("retries", 3);  
props.put("replication.factor", 3);
```

5 Silent Killers of Kafka in Production

Killer 1 — Consumer Lag

Producer: 10,000 msg/sec
Consumer: 1,000 msg/sec
Lag grows: 9,000 msg/sec → 32 million messages after 1 hour

Detection: `kafka-consumer-groups.sh --describe --group notification-group`
Fix: Add consumer instances, increase partitions, optimize consumer

Killer 2 — Rebalancing Storm

Slow consumer exceeds `max.poll.interval.ms` (5 min default)
Kafka thinks it's dead → triggers REBALANCE
ALL consumers stop processing during rebalance
Slow consumer triggers another → infinite loop

Fix: `props.put("max.poll.interval.ms", 600000); // 10 mins`
`props.put("max.poll.records", 10); // process fewer at once`

Killer 3 — Poison Pill Message

Malformed JSON in topic → consumer `NullPointerException`
Consumer retries → fails → retries → stuck forever
All messages behind it NEVER processed

Fix: Dead Letter Queue (DLQ)
After 3 retries → send to "payment-events-DLQ"
Continue with next message
Ops team reviews DLQ separately

Killer 4 — Duplicate Messages

Consumer processes payment ✔
About to commit offset → CRASHES ☒
Restarts → processes same payment AGAIN
Customer charged twice 😡

Fix: Idempotent consumer
`if (processedEventRepository.exists(event.getEventId())) return;`
`processPayment(event);`
`processedEventRepository.save(event.getEventId());`

Killer 5 — Wrong Partition Count

3 partitions, 5 consumer instances
→ 2 consumers always idle (wasteful)
→ Consumers must equal partitions for optimal throughput

Rule: $\text{partitions} = \text{peak_throughput} / \text{single_consumer_throughput}$
Can INCREASE partitions later (safe)
Cannot DECREASE (breaks ordering) → plan for growth upfront

#5 — RabbitMQ

Kafka vs RabbitMQ — The Core Difference

Kafka = YouTube
Store everything, replay anytime, multiple viewers
Built for DATA STREAMING

RabbitMQ = Pizza Delivery
Deliver to one person, delete after delivery
Built for TASK QUEUING

How RabbitMQ Works

Producer → Exchange → Queue → Consumer

4 Types of Exchange (RabbitMQ's superpower):

Exchange	Behaviour	Use Case
Direct	Routes by exact routing key match	payment.success → Success Queue
Fanout	Broadcasts to ALL bound queues	Notify ALL services simultaneously
Topic	Routes by wildcard pattern	payment.* matches all payment events
Headers	Routes by message headers	Complex routing rules

RabbitMQ vs Kafka Comparison

	RabbitMQ	Kafka
Core purpose	Task queue — deliver once	Event stream — store & replay
Message storage	Deleted after consumption	Retained for days/weeks
Replay messages	✗ No	✓ Yes
Throughput	~50,000 msg/sec	~1,000,000 msg/sec
Routing	Flexible (4 exchange types)	Only by partition key
Consumer model	Push (broker pushes)	Pull (consumer asks)

	RabbitMQ	Kafka
Best for	Task distribution, job queues	Event streaming, audit logs

When to Use Which

Use RabbitMQ when:

- ✓ One worker processes one task (send one email, generate one PDF)
- ✓ Complex routing needed (by type, priority, headers)
- ✓ Message deleted after processing (no audit trail needed)
- ✓ Low-medium throughput (< 50,000 msg/sec)

Use Kafka when:

- ✓ Multiple services need same event
- ✓ Audit trail / replay needed (RBI compliance)
- ✓ High throughput (100,000+ msg/sec)
- ✓ Event sourcing or stream processing

BFSI Decision Examples

Use Case	Choose	Why
Payment notification SMS	Kafka	Multiple consumers need same event
PDF statement generation	RabbitMQ	One worker, task done, delete
Fraud detection pipeline	Kafka	Needs replay for ML retraining
Loan approval workflow	RabbitMQ	Step-by-step, each step by one worker
Transaction audit log	Kafka	7-year retention, replay needed

Production Gotcha — Message Loss

```
// BAD – messages lost if RabbitMQ restarts
channel.queueDeclare("payments", false, false, false, null); // durable=false

// GOOD – messages survive restart (BFSI MANDATORY)
channel.queueDeclare("payments", true, false, false, null); // durable=true

AMQP.BasicProperties props = new AMQP.BasicProperties.Builder()
    .deliveryMode(2) // 2 = persistent
    .build();
```

#6 — API Gateway

The Problem Without API Gateway

Mobile app must know all 6 service URLs and ports
One service moves → app breaks
Auth logic in all 6 services → code duplication
Rate limiting in all 6 services → inconsistent

Client-side complexity = nightmare

What API Gateway Does

Single entry point for all external traffic:



Clients know ONE address: api.yourbank.com

6 Core Responsibilities

1. Routing

```
/api/accounts/** → account-service:8081
/api/payments/** → payment-service:8082
/api/loans/**    → loan-service:8083
```

2. Authentication (once, centrally)

```
@Component
public class AuthFilter implements GlobalFilter {
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String token = exchange.getRequest().getHeaders().getFirst("Authorization");
        if (!jwtUtil.isValid(token)) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }
        // Add user info to headers for downstream services
        String userId = jwtUtil.extractUserId(token);
        ServerHttpRequest mutated = exchange.getRequest().mutate()
            .header("X-User-ID", userId).build();
        return chain.filter(exchange.mutate().request(mutated).build());
    }
}
```

3. Rate Limiting

```
// Max 100 req/min per user – Payment Service never sees the flood
.filters(f -> f.requestRateLimiter(c -> c
    .setRateLimiter(redisRateLimiter())
    .setKeyResolver(userKeyResolver())))
```

4. Load Balancing

```
uri: lb://payment-service # lb = load balanced across all instances
# One instance crashes → Gateway stops sending to it automatically
```

5. SSL Termination

```
External: HTTPS (encrypted) → api.yourbank.com
Internal: HTTP (plain) → service:8081
One SSL certificate, managed in one place
```

6. Request/Response Transformation

```
Legacy service returns: {"account_id": "ACC123", "owner_name": "John"}
Mobile app expects:     {"accountId": "ACC123", "ownerName": "John"}
Gateway transforms → service unchanged, client happy
```

IMPORTANT — Two Layers of Security

API Gateway is NOT enough alone:

```
Gateway checks: Is token valid? (Authentication)
Service checks: Does this user OWN this resource? (Authorization)
```

Example:

```
User John (USR123) requests Account ACC456 (belongs to Jane USR999)
Gateway: token valid ✓ – forwards
Service: USR123 ≠ ACC456.ownerId → 403 FORBIDDEN ✗
```

Without service-level check → John accesses Jane's account!

Three things services **MUST** always validate:

1. Ownership — does this user own this resource?
2. Input validation — deep business rule validation
3. Business rules — balance, limits, account status

What API Gateway is NOT

- ✗ WRONG: Put business logic in Gateway
(balance check, eligibility, loan calculation)
- ✓ RIGHT: Gateway handles cross-cutting concerns ONLY
Business logic ALWAYS stays in services
Gateway with business logic = monolith in disguise

Popular Tools

Tool	Best For
Spring Cloud Gateway	Spring Boot microservices
Kong	Polyglot, plugin-rich
AWS API Gateway	AWS cloud
Nginx	High performance, simple routing
Istio Gateway	Kubernetes + Service Mesh

#7 — Service Mesh

The Problem at Scale

50 microservices, each needs:

- ✓ mTLS encryption to other services
- ✓ Retry failed requests
- ✓ Circuit breaking
- ✓ Load balancing
- ✓ Distributed tracing
- ✓ Metrics collection

Without Service Mesh: 50 teams × 200 lines of infrastructure code = 10,000 lines of repeated code. Different implementations. Bugs everywhere.

The Sidecar Pattern

Core concept: every service gets a proxy deployed next to it.

Every Service Pod:

[Your Service	- port 8080]
[Sidecar Proxy	- port 15001]

ALL traffic goes through the sidecar
Your service code has ZERO infrastructure logic

Payment Service → calls Fraud Service:
→ sends to localhost:15001 (its own sidecar)
→ sidecar encrypts (mTLS)
→ sidecar applies retry policy
→ reaches Fraud Service's sidecar
→ sidecar decrypts → forwards to Fraud Service

Your code: zero involvement. Zero lines of infrastructure code.

Architecture

CONTROL PLANE (Istio)
Pilot → routing rules
Citadel → certificate management
Galley → configuration
↓ pushes config to all proxies

DATA PLANE
[Account Svc + Envoy] ↔ [Payment Svc + Envoy] ↔ [Fraud Svc + Envoy]
Actual traffic flows through Envoy sidecars

What Service Mesh Gives You For Free

Automatic mTLS — zero code:

```
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
spec:
  mtls:
    mode: STRICT # all internal traffic encrypted ✓
# One YAML → all 50 services encrypted. No code changes.
```

Canary Deployment — pure config:

```
# Send 10% traffic to new version, 90% to stable
- destination:
  host: payment-service
  subset: v1
  weight: 90
- destination:
  host: payment-service
  subset: v2
  weight: 10
```

Circuit Breaking — no Resilience4j needed:

```
outlierDetection:
  consecutiveErrors: 5    # after 5 errors
  interval: 30s          # within 30 seconds
  baseEjectionTime: 30s  # remove from pool for 30 seconds
```

Automatic Observability:

Every request automatically gets:

- ✓ Latency tracked
- ✓ Error rate tracked
- ✓ Distributed trace created
- ✓ Visible in Grafana/Jaeger

Zero logging code needed in your services

API Gateway vs Service Mesh — Clear Difference

	API Gateway	Service Mesh
Sits between	External clients & services	Internal services only
Traffic direction	North-South (in/out)	East-West (internal)
Auth type	User auth (JWT)	Service identity (mTLS certs)
Instances	One gateway	Sidecar per service
Examples	Spring Cloud Gateway, Kong	Istio, Linkerd

```
Internet → [API Gateway] → Service A → [Service Mesh] → Service B
           ↑                               ↑
       North-South                       East-West
    (external traffic)                 (internal traffic)
```

They complement each other — you need BOTH in production.

When Do You Need Service Mesh?

```
5 services    → Don't need it (overkill)
20 services   → Consider it
50+ services  → Absolutely need it
```

Signs you need it:

- Teams implement security inconsistently
- Can't see what's happening between services
- SSL/mTLS setup is painful
- Canary deployments are manual and risky
- Debugging latency issues is impossible

Master Summary — All 7 Communication Patterns

Pattern	Problem Solved	Key Decision
Sync vs Async	When to wait vs fire-and-forget	Does caller need result immediately?
REST	Standard service communication	Public APIs, external-facing
gRPC	High-performance internal comms	Internal services, high throughput
Kafka	Async event streaming	Multiple consumers, audit, replay
RabbitMQ	Task queuing	One worker per task, complex routing
API Gateway	Single entry point for clients	External traffic management
Service Mesh	Internal service infrastructure	20+ services, zero-trust security

Interview Cheat Sheet

Q: When do you use Kafka vs RabbitMQ?

Kafka for events that multiple services need, need replay, or need audit trail (BFSI standard). RabbitMQ for task queues where one worker handles one job and deletes it after.

Q: What does API Gateway do?

Single entry point for external traffic. Handles routing, authentication, rate limiting, load balancing, SSL termination. Business logic never goes in Gateway.

Q: Difference between API Gateway and Service Mesh?

Gateway handles north-south traffic (external to internal). Service Mesh handles east-west traffic (service to service). Gateway does user auth (JWT), Mesh does service identity (mTLS). You need both.

Q: What are the 4 REST killers in production?

No connection pooling (new TCP per request), no timeouts (thread pool exhaustion), wrong HTTP status codes (200 for errors), no correlation IDs (impossible to trace failures).

Q: Why gRPC over REST for internal services?

3–5x smaller payload (Protobuf vs JSON), 5–10x faster, strict contracts (.proto), supports streaming (4 types). REST still used for external/public APIs.

Q: What is the sidecar pattern?

A proxy deployed next to every service. Handles mTLS, retries, circuit breaking, metrics — all without any code in the service itself. Foundation of Service Mesh.

Module 1 of 6 complete / Next: Module 02 — Data Patterns (15 patterns)

After all 6 modules: 30+ Scenario-Based Interview Questions